

Table of contents

Interprétation Abstraite	2
Introduction	2
Problème de correction des programmes	2
L'importance de la sémantique : un exemple	2
Indécidabilité et approximation	5
Sémantique Collectrice (Collecting Semantics)	6
Principe général	6
Langage étudié	6
Graphe de Contrôle (Control Flow Graph)	6
Définition de la Sémantique Collectrice	6
Équation à point fixe	7
Théorie du Point Fixe	7
Définitions fondamentales	7
Bornes supérieures et inférieures	8
Exemples de treillis	8
Fonctions monotones et Scott-continues	9
Théorèmes fondamentaux	10
Algorithme de calcul du point fixe	10
Produit de treillis	11
Connexion de Galois	11
Propriétés de sécurité et de vivacité	11
Observateur de sécurité	11
Problème d'indécidabilité	12
Méthode d'analyse	12
Définition de la Connexion de Galois	12
Application à l'interprétation abstraite	13
Correction de l'analyse abstraite	13
Exemple complet d'analyse	13
Programme exemple	13
Construction du CFG	14
Équations de sémantique collectrice	14
Calcul itératif	15
Analyse avec propriété de sécurité	15
Comparaison des méthodes de vérification	15
Conclusion	16

Interprétation Abstraite

Introduction

L'interprétation abstraite est une méthode formelle pour analyser les propriétés des programmes informatiques. Elle repose sur l'idée d'approximer la sémantique d'un programme de manière conservative.

Problème de correction des programmes

Plusieurs approches existent :

- **Logique de Hoare** (sémantique axiomatique)
 - Raisonnement déductif
 - Solveurs SMT (Satisfiability Modulo Theories)
- **Interprétation abstraite**

L'objectif est de déterminer si un programme satisfait certaines propriétés de sécurité.

L'importance de la sémantique : un exemple

Pour illustrer comment la sémantique exacte d'un langage influence l'exécution, comparons deux programmes équivalents en C et en Ada.

Programme C :

```
int i, a;  
a = 5;  
for (i = 0; i < a; i++) {  
    a--;  
    printf("i = %d", i);  
}
```

Résultat : i = 0, i = 1, i = 2

Programme Ada :

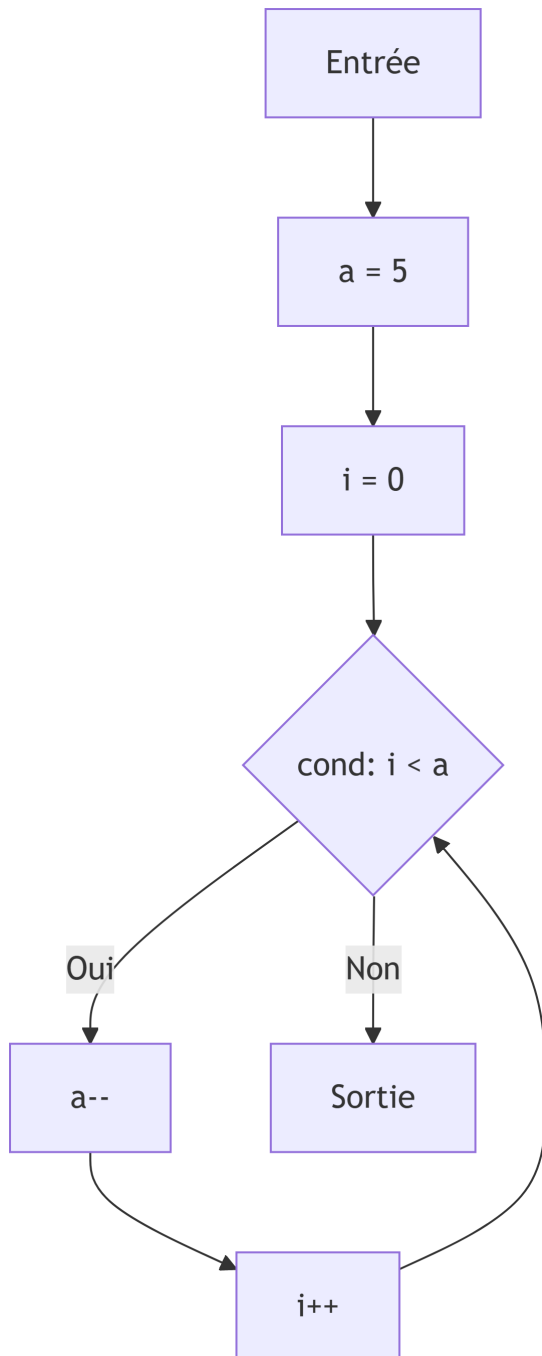
```
A: integer;  
begin  
  a := 5;  
  for I in 0..A loop  
    A := A - 1;  
    Put("i="); Put(I);  
  end loop;  
end;
```

Résultat : i = 0, i = 1, i = 2, i = 3, i = 4, i = 5

Explication : En Ada, la borne supérieure de la boucle `for` est évaluée une seule fois avant l'entrée dans la boucle (copie de la valeur de `A`), alors qu'en C la condition `i < a` est réévaluée à chaque itération avec la valeur courante de `a`.

Modélisation par automate

Nous pouvons représenter le flux de contrôle de ces programmes à l'aide d'automates. Voici l'automate correspondant au programme C :



Pour le programme Ada, l'automate serait différent car la valeur de **A** dans la condition est fixée avant la boucle.

Indécidabilité et approximation

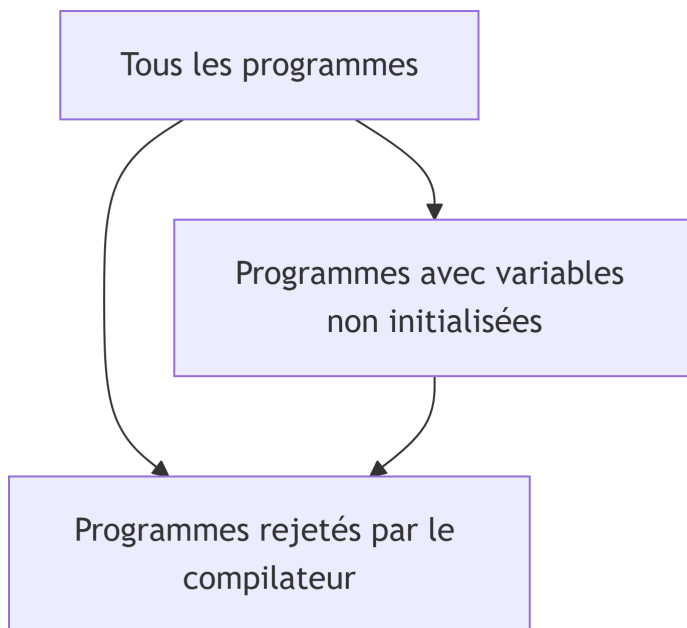
Considérons l'exemple Java suivant :

```
{  
  int a;  
  for (int i = 0; i <= a; i++) {  
    // blablabla  
  }  
}
```

Le compilateur Java refusera de compiler ce code ou émettra un avertissement sérieux : “*Variable ‘a’ might not have been initialized*”.

Problème : La détection des variables non initialisées est un problème **indécidable** (non calculable).

Le compilateur donne donc une **approximation** – plus précisément une **sur-approximation** (over-approximation).



L'ensemble des programmes rejetés est **plus grand** que l'ensemble des programmes réellement problématiques. C'est une **approximation conservative** :

- Si le programme n'est **pas rejeté**, on est sûr qu'il n'a **pas** de problème de variable non initialisée.
- S'il est rejeté, on ne **sait pas** s'il a réellement un problème (faux positif possible).

Sémantique Collectrice (Collecting Semantics)

Principe général

L'analyse doit être **conservative** : elle calcule une approximation conservative (abstraction) telle que la réponse soit sûre.

- Si l'analyse ne **rejette pas** le programme $\rightarrow$ sa correction est **garantie**
- Si l'analyse **rejette** le programme $\rightarrow$ réponse "peut-être" (il peut y avoir une erreur ou non)

Langage étudié

Le langage utilisé dans ce chapitre est le même qu'au premier chapitre :

- Expressions entières et booléennes
- Instructions : **if**, **while**, affectations
- **Interdiction** des appels de fonctions (récursifs ou non)

Graphe de Contrôle (Control Flow Graph)

Soit P un programme. Un **graphe de contrôle** pour P est un automate interprété (S, T, s_0, Var) où :

- S : ensemble fini d'états
- $s_0 \in S$: état initial
- Var : ensemble des variables utilisées dans P
- $T \subseteq S \times (A(Var) \cup B(Var)) \times S$: ensemble des transitions
 - $A(Var)$: ensemble des affectations $x := expr$ où $x \in Var$ et $expr$ est une expression sur Var
 - $B(Var)$: ensemble des conditions (expressions booléennes sur Var)

La construction d'un CFG se fait récursivement sur chaque instruction de P .

Définition de la Sémantique Collectrice

La **sémantique collectrice** (Floyd-Hoare) associe à chaque état (point de contrôle) du CFG l'ensemble de toutes les valeurs que les variables peuvent prendre lors de l'exécution.

Soit α un point de contrôle. On note $V(\alpha)$ cet ensemble :

- Pour une variable x , $V(\alpha)(x)$ est l'ensemble des valeurs possibles que x peut prendre au point α

Règles de calcul

1. **Transition par affectation** $x := expr$ de A à B :

$$V(B) = V(A)[x := expr]$$

où $V(B)$ prend toutes les valeurs de $V(A)$ sauf pour $V(B)(x)$ qui prend les valeurs obtenues en évaluant $expr$ sur $V(A)$.

2. **Transition par condition** $cond \in B(Var)$ de A à B :

$$V(B) = V(A) \cap [cond]$$

où $V(B)$ prend toutes les valeurs de $V(A)$ qui satisfont la condition $cond$.

3. **Confluence** (plusieurs transitions vers C) :

- Si on a des transitions de A à C et de B à C
- On calcule d'abord les effets : $V'(A)$ et $V'(B)$
- Puis : $V(C) = V'(A) \cup V'(B)$

Équation à point fixe

Les programmes avec boucles créent des cycles dans le CFG. Considérant V comme un vecteur $(V(A), V(B), \dots)$, on obtient une équation où V dépend de lui-même :

$$V = \Psi(V)$$

où Ψ est la fonction dérivée du graphe de contrôle de P .

C'est une **équation à point fixe** : la solution V de la sémantique collectrice est solution de $X = \Psi(X)$.

Théorie du Point Fixe

Définitions fondamentales

Ensemble partiellement ordonné (poset) : $(X, \leq)$ où $\leq$ est une relation réflexive, anti-symétrique et transitive.

Treillis (Lattice) : poset $(X, \leq)$ tel que pour toute paire $A, B \in X$:

- L'infimum $\inf(A, B)$ existe dans X
- Le supremum $\sup(A, B)$ existe dans X

Treillis complet : treillis $(X, \leq)$ tel que tout sous-ensemble de X a un infimum et un supremum dans X .

Note : Tout treillis fini non vide est complet. Seuls les treillis infinis peuvent être incomplets.

Bornes supérieures et inférieures

Soit $(X, \leq)$ un poset, $Y \subseteq X$:

- **Borne supérieure** de Y : $u \in X$ tel que $\forall y \in Y, y \leq u$
- **Borne inférieure** de Y : $l \in X$ tel que $\forall y \in Y, l \leq y$
- **Supremum** $\sup Y$: plus petite borne supérieure de Y
- **Infimum** $\inf Y$: plus grande borne inférieure de Y

Exemples de treillis

1. $(\mathbb{Z}, \leq)$: treillis mais **pas complet**

2. $(\mathcal{P}(\mathbb{Z}), \subseteq)$: treillis **complet**

- $\inf(A, B) = A \cap B$
- $\sup(A, B) = A \cup B$
- $\sup Y = \bigcup_{S \in Y} S$
- $\inf Y = \bigcap_{S \in Y} S$

3. **Intervalles** :

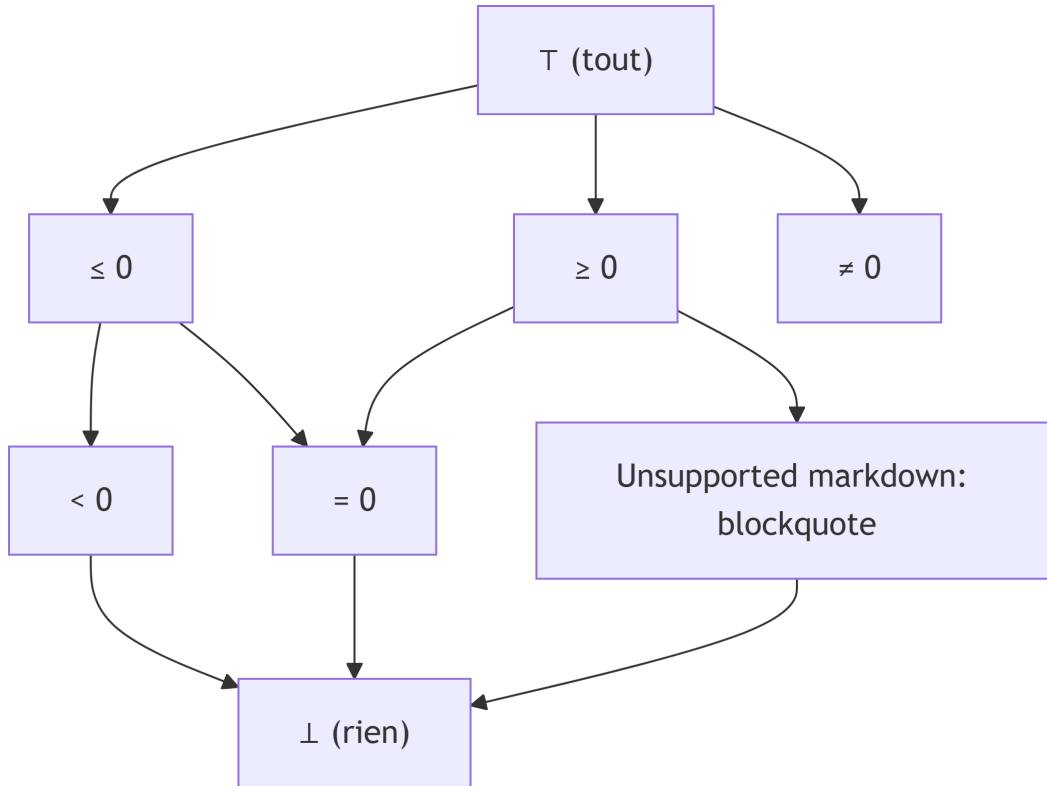
$$\mathcal{I} = \{[a, b] \mid a, b \in \mathbb{Z}, a \leq b\} \cup \{(-\infty, a] \mid a \in \mathbb{Z}\} \cup \{[a, +\infty) \mid a \in \mathbb{Z}\} \cup \{\perp, (-\infty, +\infty)\}$$

avec un ordre représentant la “précision” (voir exercices) : treillis complet.

4. **Signes** :

$$\mathcal{S} = \{< 0, \leq 0, > 0, \geq 0, = 0, \neq 0, \perp, \top\}$$

avec un ordre représentant la précision : treillis complet.



Fonctions monotones et Scott-continues

Fonction monotone : Soient $(E, \leq)$ et $(F, \leq)$ deux treillis. $f : E \rightarrow F$ est **monotone** (non-décroissante) ssi :

$$\forall x, y \in E, x \leq y \Rightarrow f(x) \leq f(y)$$

Fonction Scott-continue : Soient $(E, \leq)$ et $(F, \leq)$ deux treillis complets. $f : E \rightarrow F$ est **Scott-continue** ssi pour toute suite non-décroissante $U_0 \leq U_1 \leq \dots \leq U_n \leq \dots$ dans E :

$$\sup\{f(U_n), n \geq 0\} = f(\sup\{U_n, n \geq 0\})$$

Propriétés :

- Si f est continue, alors f est monotone
- La composition préserve la monotonie
- La composition préserve la Scott-continuité

Théorèmes fondamentaux

Théorème de Knaster-Tarski

Soit $(E, \leq)$ un treillis complet. Soit $f : E \rightarrow E$ une fonction monotone.

L'ensemble des points fixes de f , $FP = \{x \in E \mid f(x) = x\}$, est un treillis complet. En particulier, f admet un **plus petit point fixe** (least fixed-point) et un **plus grand point fixe** (greatest fixed-point).

Théorème de Kleene

Soit $(E, \leq)$ un treillis complet. Soit $f : E \rightarrow E$ une fonction Scott-continue.

Le plus petit point fixe de f est égal à :

$$\text{lfp}(f) = \sup\{f^n(\perp), n \geq 0\}$$

où $\perp = \inf E$ et f^n est définie inductivement par :

- $f^0(x) = x$
- $f^{n+1}(x) = f(f^n(x))$

Propriété : Soient $(E, \leq)$ et $(F, \leq)$ deux treillis complets. Soient $f, g : E \rightarrow F$ deux fonctions continues telles que $\forall x \in E, f(x) \leq g(x)$. Alors :

$$\text{lfp}(f) \leq \text{lfp}(g)$$

Algorithme de calcul du point fixe

Le théorème de Kleene fournit un algorithme pour approximer le plus petit point fixe :

```
X :=  
repeat  
  X' := f(X)  
until X' = X  
return X
```

Attention : Si le treillis est infini, cet algorithme peut ne pas terminer.

Produit de treillis

Lemme : Soient $(E, \leq)$ et $(F, \leq)$ deux treillis complets. On définit l'ordre sur $E \times F$ par :

$$\forall (a, b), (c, d) \in E \times F, (a, b) \leq (c, d) \text{ ssi } a \leq c \text{ et } b \leq d$$

Alors $(E \times F, \leq)$ est un treillis complet.

Application : Pour un programme avec $\#v$ variables et $\#cp$ points de contrôle, l'équation $X = \Psi(X)$ s'exprime sur le treillis $\mathcal{J}^{\#v \times \#cp}$ (si on utilise les intervalles).

Connexion de Galois

Propriétés de sécurité et de vivacité

Les spécifications d'un programme peuvent être divisées en deux types :

- **Sécurité (Safety) :** “Rien de mauvais ne se produira jamais”
 - Violée en temps fini
 - Exemple : pas de division par zéro
- **Vivacité (Liveness) :** “Quelque chose de bon finira par se produire”
 - Exemple : terminaison

Nous nous concentrons sur les **propriétés de sécurité**.

Observateur de sécurité

Une propriété de sécurité peut être exprimée à l'aide d'un **observateur** (automate) qui :

- Lit les valeurs du programme
- Passe à un état ERROR lorsque la propriété est violée

On peut transformer le CFG pour que la propriété de sécurité soit satisfaite ssi l'état ERROR est inaccessible.

Problème d'indécidabilité

L'accessibilité d'un point de contrôle n'est **pas décidable** en général. On utilise donc la sémantique collectrice sur un domaine abstrait pour obtenir une réponse conservative.

Analyse conservative :

- Réponse “OK” (pas d’erreur) → **garantie** de correction
- Réponse “Je ne sais pas” → erreur possible (ou non)

Méthode d'analyse

1. **Choisir un domaine abstrait** (signes, intervalles, ...)
2. **Exprimer la propriété de sécurité comme un observateur**
3. **Transformer le programme et l'observateur** en un CFG avec état ERROR
4. **Exprimer sous forme d'équations à point fixe**
5. **Calculer (essayer) le plus petit point fixe**
 - Si l'état ERROR est atteint (valeur $\neq \perp$) → “Je ne sais pas”
 - Si un point fixe est atteint sans ERROR → “OK”
 - (Cette étape peut ne pas terminer)

Définition de la Connexion de Galois

Une **connexion de Galois** entre :

- Un treillis complet concret $(E, \leq)$
- Un treillis complet abstrait $(F, \preceq)$

est une paire de fonctions $\alpha : E \rightarrow F$ (abstraction) et $\gamma : F \rightarrow E$ (concrétisation) telle que :

$$\forall x \in E, \forall y \in F, \alpha(x) \preceq y \iff x \leq \gamma(y)$$

Propriétés

1. $\gamma \circ \alpha$ est **extensive** : $\forall x \in E, x \leq \gamma(\alpha(x))$
2. $\alpha \circ \gamma$ est **intensive** : $\forall y \in F, \alpha(\gamma(y)) \preceq y$
3. α et γ sont monotones
4. α est Scott-continue

Application à l'interprétation abstraite

Dans notre contexte :

- **Treillis concret** : $\mathbb{Z}^{\#v}$ (valeurs des variables)
- **Treillis abstrait** : $\mathcal{I}^{\#v}$ (intervalles)
- **Abstraction** α : transforme les ensembles de valeurs en intervalles
- **Concrétisation** γ : transforme les intervalles en ensembles de valeurs

La sémantique collectrice s'exprime côté concret : $V = \Psi(V)$

Le calcul de point fixe se fait côté abstrait : $W = \Psi_{\text{abst}}(W)$

On peut prendre :

$$\Psi_{\text{abst}} = \alpha \circ \Psi \circ \gamma$$

À condition que Ψ et γ soient continues.

Correction de l'analyse abstraite

En calculant $\text{lfp}(\Psi_{\text{abst}})$, on obtient une **sur-approximation** (over-approximation), donc une réponse conservative. On peut prouver que :

$$\gamma(\text{lfp}(\Psi_{\text{abst}})) \geq \text{lfp}(\Psi)$$

Parfois, on utilise une fonction encore plus abstraite ξ telle que $\forall y \in F, \Psi_{\text{abst}}(y) \leq \xi(y)$ pour accélérer le calcul et assurer la terminaison. Dans ce cas aussi, on obtient une sur-approximation :

$$\gamma(\text{lfp}(\xi)) \geq \text{lfp}(\Psi)$$

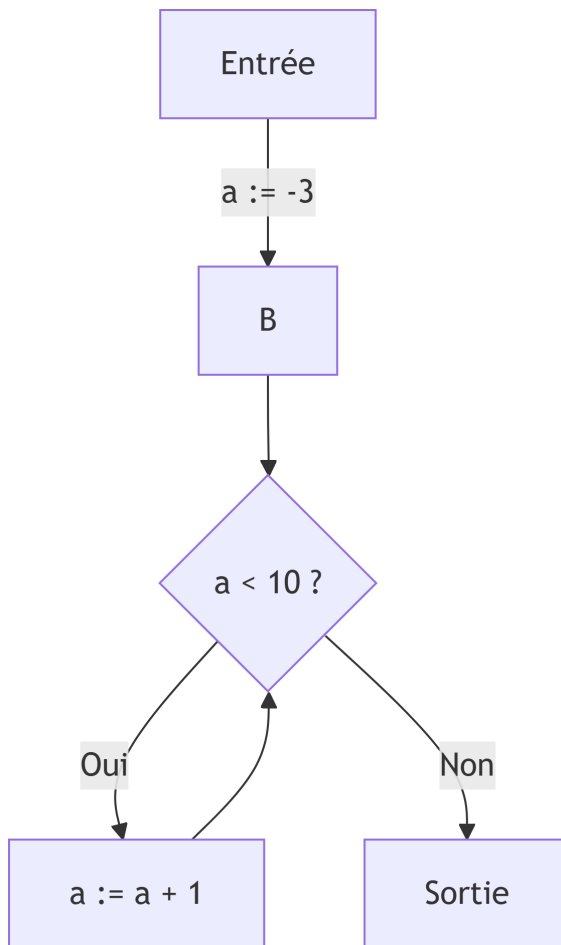
Exemple complet d'analyse

Programme exemple

Considérons le programme simple :

```
a = -3
while a < 10 do
  a = a + 1
done
```

Construction du CFG



Équations de sémantique collectrice

Sur le domaine des signes $\mathcal{S}$:

- $V(A) = \top$
- $V(B) = V(A)[a := -3] \cup V(D)[a := a + 1]$
- $V(C) = V(B) \cap [a < 10]$
- $V(D) = V(B) \cap [a \geq 10]$

On cherche $V = \phi(V)$ où $V \in \mathcal{S}^4$.

Calcul itératif

1. Initialisation : $V^0 = (\perp, \perp, \perp, \perp)$
2. Itération 1 : $V^1 = \phi(V^0)$
 - $V^1(A) = \top$
 - $V^1(B) = \top[a := -3] \cup \perp[a := a + 1] = \{a = -3\} \cup \perp = \{a = -3\}$
 - $V^1(C) = \{a = -3\} \cap [a < 10] = \{a = -3\}$
 - $V^1(D) = \{a = -3\} \cap [a \geq 10] = \perp$
3. Itération 2 : $V^2 = \phi(V^1)$
 - Continuer jusqu'à convergence...

Analyse avec propriété de sécurité

Supposons qu'on veuille vérifier que a ne devient jamais positif. On ajoute un observateur qui passe en ERROR si $a > 0$.

L'analyse abstraite déterminera si l'état ERROR est atteignable.

Comparaison des méthodes de vérification

Méthode	Système	Propriété	Technique
Logique de Hoare	Programme	Pré/post-conditions	Preuves déductives
SMT Solving	Formule logique	Formule à vérifier	Solveurs automatiques
Interprétation abstraite	Programme	Propriété de sécurité (observateur)	Point fixe sur domaine abstrait
Model Checking	Automate fini	Logique temporelle (LTL, CTL)	Exploration d'états

Pour l'interprétation abstraite, on vérifie que l'ensemble des traces du système est inclus dans l'ensemble des traces de la propriété :

$$\text{Tr}(\text{Système}) \subseteq \text{Tr}(\text{Propriété})$$

Conclusion

L'interprétation abstraite est une méthode puissante pour l'analyse statique de programmes. Elle permet de vérifier des propriétés de sécurité de manière conservative, avec des garanties formelles grâce à la théorie des treillis complets et des connexions de Galois.

Les points clés à retenir :

1. L'analyse est **conservative** (sûre mais pas complète)
2. Elle repose sur la **sémantique collectrice** et les **équations à point fixe**
3. La **théorie des treillis** et les **connexions de Galois** fournissent le fondement mathématique
4. Le choix du **domaine abstrait** influence la précision et l'efficacité de l'analyse